Interfaces or Bodies? A Preregistered Test of ε-Sufficient Context for Language-Model Code Editing in Go

Jude Castiel Johansen
Shockware Limited, independent researcher
castiel@shockware.co.uk ORCID 0009-0002-5644-3073

***Abstract*—Coding agents decide, many times a day, how much of a repository to show a language model, and the decision is made by heuristic or by end-task scores that confound localisation, context size, and model quality. A recent note defines *ε-sufficient context*: a context is sufficient for an editor when the editor's success on it is within ε of its success on the best context in a stated family, and proves that under a bounded reader this property is not closed under superset. We propose a preregistered, paired, multi-model study that operationalises the definition as a non-inferiority test. Holding localisation fixed at the gold write-set on at least 300 post-cutoff fail-to-pass instances mined from five to eight Go repositories, we compare (H2) a compiler-computed context consisting of the write-set and the exported interfaces of its dependencies against the same package set with full bodies and against the transitive slice; (H1) we estimate each model's degradation curve by padding a fully covered base context to fixed absolute loads up to 200k tokens with three padding types; (H3) we measure, descriptively, whether showing the forward slice reduces breakage of passing tests; and (H4) we report tokens per arm. Coverage is defined operationally per arm by checking that the gold patch is expressible from the context. Three models, three samples per cell on the primary arms and ten on a cheap dense model (temperature zero on open-weights models; the frontier models' sampling parameters cannot be set, so they run at vendor defaults), paired analysis with bootstrap and Wilson intervals and Holm correction. The instances, slices, and contexts are released as SuffBench v0.1. A 24-instance pilot on one repository (2,360 verdicts, three Claude models, about $61) has been run and published: the write-set plus dependency interfaces had a higher pass rate than the same packages with bodies ($+0.04$ [$+0.01$, $+0.08$]) and than the write-set alone ($+0.07$); padding to 200k tokens produced no consistent sign across models; the strongest model solved 14 of 24 tasks from the issue text alone; and 25% of cells were non-unanimous across samples. The pilot fixes the margin, adds a mandatory task-only screen, extends the load range to one million tokens, and declares a flat degradation curve an admissible outcome.**

I. Introduction

Every system that uses a language model to edit code must choose which parts of the repository to place in the model's context. The choice is consequential in two directions. Too little, and the model lacks a fact it needs; too much, and its accuracy falls with input length even when the needed facts are present [1], [2], [3]. Current practice chooses by retrieval heuristics or by the agent's own exploration [4], [5], and evaluates the choice only through the end-task pass rate, which confounds whether the right place was found, how much was shown, and how good the model is.

A recent definitions note [6] separates these. A change request is a contract (Δ, D, I, W): requested behaviour, scope of permitted behavioural change, invariants, and a write-set. A context K is *ε-sufficient* for an editor A when A's success on K, judged by an oracle of tests, is within ε of its success on the best context in a stated candidate family. Under a stylised model in which success factors into coverage of needed facts and a non-increasing function of load, the note proves, with a Lean 4 mechanisation, that sufficiency is not closed under superset: adding irrelevant material to a sufficient context can make it insufficient, and the admissible tolerance grows with the amount added. The note also states three properties a language could enforce so that the sufficient context becomes a graph query: dependencies matter only through their declared interfaces (P1); dependents rely only on what interfaces pin down (P2); and the context is computable from the dependency graph before the edit (P3). A mutation study on one Go repository found that interface-preserving faults were caught by tests outside the changed package in about one case in twenty-three, concentrated in shared helper packages.

None of that is evidence about real editors. A 24-instance pilot [7], described in Section V, gives first estimates and shaped this design; it is exploratory and is not reused. This report proposes the study that would supply confirmatory evidence, designed to be cheap enough for an independent researcher and powered enough to be worth running. Its central question is P1 as an experiment: does an editor given only the interfaces of what the write-set depends on do as well as one given the bodies, and as well as one given the transitive slice? Framed as non-inferiority, this is exactly the ε-sufficiency definition. Around it we estimate the degradation curve per model at verified fixed coverage, measure the forward slice's effect on regressions descriptively, and report the token cost of every arm.

A. Relation to recent work

Several 2026 measurements bear directly on the design and we build on them rather than claim to precede them. SamBodden [8] holds localisation fixed on SWE-bench Verified and varies the representation of surrounding Python code, finding that signatures and skeletons serve about as well as source; his preregistered superiority test failed to find a difference, which is precisely the gap a powered non-inferiority

test fills. Mohammadi et al. [9] show across seven models that availability of needed facts decides success and their position does not. Du et al. [3] show that length alone hurts even under perfect retrieval and neutral padding. Xue [10] reports large drops from short to long contexts on two auditing tasks. Alonso et al. [11] reduce agent regressions on SWE-bench Verified from about six percent to under two with a graph-based impact map. Jia, Barr and Mechtaev [12] search for minimal sufficient context on the editor side. Hierarchical context pruning [13] found that pruning function bodies in dependent files did not significantly reduce completion accuracy. Bjarnason et al. [14] quantify run-to-run randomness in agent evaluation, which bounds what any paired comparison can resolve and motivates our pilot.

What remains open, and what this study measures: the interfaces-versus-bodies question as a powered non-inferiority test in a statically typed language with type-graph-derived interfaces; a per-model degradation curve on repository editing with verified coverage and intervals; the forward slice's effect in a single-shot setting stratified by whether the write-set is a shared helper; and a dataset whose relevance labels come from test verdicts rather than from workflow signals or gold-patch files [15], [16].

II. Hypotheses and estimands

All contrasts are paired by instance. p_X denotes the population-average probability, over instances and samples, that a patch produced from context X passes the fail-to-pass tests (Δ-pass). Where stated, the outcome is instead the conjunction Δ-pass and frame-pass (all pass-to-pass tests still pass).

- **H2 (primary): interface sufficiency.** $p_{\mathrm{A1}} \geq p_{\mathrm{A1b}} - \varepsilon$ and $p_{\mathrm{A1}} \geq p_{\mathrm{A2}} - \varepsilon$, where A1 is write-set bodies plus interfaces of direct dependencies, A1b the same package set with bodies, and A2 the transitive slice. Non-inferiority margin ε fixed after the pilot (Section V); primary candidate $\varepsilon = 0.075$, secondary $\varepsilon = 0.05$. Estimand: population-average over instances. Per-instance ε-sufficiency is estimated only on the subset with three samples per cell.
- **H1: degradation.** At fixed, verified coverage, Δ-pass is non-increasing in absolute load. Tested as A1 versus A1 padded to 200k, 400k and 950k tokens (superiority, one-sided), separately for three padding types; the full curve at {base, 64k, 200k, 400k, 700k, 950k} is reported with intervals. A flat curve is an admissible and reportable outcome; the pilot found no consistent sign up to 200k.
- **H3 (descriptive): forward slice.** The rate of Δ-pass-and-frame-pass for A1F (A1 plus interfaces and test names of the forward slice) versus A1, overall and stratified by whether the write-set is a shared helper (number of dependents at or above the median). Reported with intervals; not a confirmatory test unless shared-helper oversampling yields at least 20 frame failures.
- **H4: cost.** Tokens, latency, and price per arm; the ratio of A1 to A2 and to lexical retrieval at matched and at large budgets.
- **Control: parametric memory.** $p_{\mathrm{A}\emptyset}$, task text only, estimates how often the model succeeds with no repository context. In the pilot the strongest model solved 14 of 24 instances this way. The task-only arm is therefore run first, on the strongest evaluated model with $k = 3$; any instance solved at least once is excluded from the confirmatory set and reported separately.

III. Instances: SuffBench v0.1

Sources. Five to eight actively developed, single-module Go repositories with a green Linux baseline, selected before mining by stars, activity, and test density; the GitHub CLI will be one. The Multi-SWE-bench harness and container recipes [17] are reused.

Mining. For each commit after the latest documented training cutoff of any evaluated model, touching both non-test and test files: build at parent and child; the child's test files are applied to the parent; a test is *fail-to-pass* iff its package compiles at the parent with the child's tests and it fails there and passes at the child; the full suite must be green at the child. Instances whose fail-to-pass tests reference identifiers introduced by the gold patch are excluded unless the new signatures are supplied as specification, in which case the instance is flagged. The write-set is the set of changed packages; the interface write-set is computed by exported-API diff.

Task text. The linked issue body where one exists; otherwise a behaviour-only specification rewritten from the PR body by a language model with all identifiers and file names removed, hand-checked by the author. The source is recorded and results are reported by source.

Screening. Every instance is screened manually for under-specification and leakage, about ten minutes each, following SWE-bench Verified's protocol, and automatically by the task-only control above. The pilot's syntactic leak rule (tokens introduced by the gold patch into the write-set that the test diff references) is applied first; it rejected none of 25 candidates while manual inspection vetoed one, so both are required. Target: at least 300 confirmatory instances after screening, with two-package write-sets oversampled to at least 25%.

Slices and checks. For each instance: backward and forward slices from `go list -deps -test`; a coverage slice from per-package coverage of the verdict tests on the gold tree; a mutation slice using body-only, interface-preserving operators; a closure check that the write-set with its backward and forward slices covers the import closure of the executed test binaries; and, per arm, an *expressibility* check that every identifier, declaration, and import the gold patch references is present in the context. Instances where the gold patch adds imports outside the parent's dependency graph are flagged.

Flakiness. Every test that fails on any run is rerun three times on the clean tree; those failing on the clean tree are excluded and listed.

Arm	Content	Typical load
A$\emptyset$	task text only	<2k
A0	W bodies	3k
A1	W bodies + interfaces of deps(W)	15k
A1b	W bodies + bodies of deps(W)	40k
A1F	A1 + interfaces and test names of $F(W)$	25k
A2	W + bodies and tests of $B(W) \cup F(W)$	150k
A4s	lexical retrieval at A1's budget	15k
A4l	lexical retrieval at a large budget	150k
P_{64}..P_{950}	A1 padded to 64k, 200k, 400k, 700k, 950k	as named

TABLE I

ARMS. PADDING TYPES: SAME-REPOSITORY PACKAGES OUTSIDE THE TEST-BINARY CLOSURE WITH A LEXICAL-SIMILARITY CEILING; CROSS-REPOSITORY GO; NATURAL-LANGUAGE PROSE. LOADS ARE MEDIANS FROM THE PILOT ON THE GITHUB CLI; ARMS ABOVE 200K RUN ONLY ON MODELS WITH WINDOWS OF ONE MILLION TOKENS.

Release. Under embargo until Stage 1 submission; then on Zenodo with the datasheet required by the public SuffBench specification [18].

IV. Design

Arms. Table I. Localisation is fixed at the gold write-set W in all arms but the retrieval baselines. Contexts are serialised deterministically with W last, immediately before the task.

Editor protocol. Single shot: task text and context in, edits out in a search-and-replace block format, applied with a strict matcher. One format-repair turn is allowed only when a block fails to apply; no test feedback is ever given. Open-weights models run at temperature zero; the frontier models' sampling parameters (temperature, top-p, top-k) are not settable, so each of their samples is an independent call at vendor defaults and run-to-run variation is estimated from the samples. Ten samples per cell on the dense cheap model for every arm; three per cell on the other models on A1, A1b, A2 and the padded arms; one elsewhere. Identical prompts share a prompt cache and identical patches share a verdict, which is what made the pilot cost $61 for 2,360 verdicts. Provider and quantisation are pinned and recorded.

Models. A dense cheap model (Claude Haiku 5.5 class) on all arms and all instances with ten samples per cell; one frontier model via its vendor API on A1, A1b, A2, P_{200}, P_{950} and A$\emptyset$ over a stratified random 100-instance subset; one open-weights model via a hosted API on all arms at temperature zero. Documented training cutoffs are recorded and every instance postdates them.

Verification. Stage one runs the fail-to-pass tests; stage two runs the full suite only for patches that pass stage one. Outcomes: applied, Δ-pass, frame-pass, tokens in and out, latency.

V. Analysis plan

Pilot. Twenty-four instances mined from 282 GitHub CLI commits by the rules above (every rejection logged), three models, 2,360 verified samples, published with all data [7]. Findings that bind this design: (i) A1 had a higher Δ-pass rate than A1b by $+0.042$ [$+0.012$, $+0.075$] and than W by $+0.067$ [$+0.021$, $+0.133$] on Haiku 5.5 over 24 instances, with no majority-outcome disagreement between A1 and A1b; (ii) padding A1 to 200k changed Δ-pass by -0.03 [-0.15, $+0.09$] on Haiku and by $+0.04$ to $+0.07$ with intervals touching zero on Sonnet 5.5 and Opus 5; (iii) Sonnet solved 14 of 24 instances from the task text alone; (iv) 25% of Haiku cells and 11% of Sonnet cells were non-unanimous across samples, with the minority outcome holding about 30% of samples; (v) nine instances were ceilings on which the write-set alone always passed. Pilot data are exploratory and are not reused.

Margin. The pilot shows that cell non-unanimity is the wrong quantity for a margin: at 0.25 it would imply $\varepsilon \geq 0.5$. The margin is instead set from the per-instance pass-rate differences: ε is the smallest value in $\{0.05, 0.075, 0.10\}$ such that, with $n = 300$ and the pilot's observed standard deviation of per-instance A1$-$A1b differences (0.11 on Haiku), a paired one-sided test has at least 80% power at a true difference of zero; on the pilot numbers this is $\varepsilon = 0.05$ for the dense model (n required $\approx 6.18\,\sigma^2/\varepsilon^2 = 30$) and $\varepsilon = 0.075$ for models with $k = 3$, whose per-instance differences are coarser ($\sigma \approx 0.25$, $n \approx 69$). These values are fixed here before any confirmatory run. Sensitivity with π_d as in the original plan: power at $\varepsilon = 0.075$, $\pi_d = 0.2$, $n = 300$ is 0.90.

Tests. H2 uses an intersection–union non-inferiority test: both comparisons must hold, each judged by the lower bound of a paired interval for the mean per-instance difference (instance-level bootstrap, 2,000 resamples, with the paired score interval for correlated proportions reported alongside) exceeding $-\varepsilon$. H1 uses one-sided paired tests of superiority with the same interval. Holm correction is applied across the family of confirmatory tests per model (two for H2, three per padding type for H1). Intervals on all proportions are Wilson intervals; bootstrap over instances is reported alongside. Pass rates across padding levels are compared only through paired differences, since levels are built for different instance subsets.

No interim stopping. The sample size is fixed in advance.

Reporting. All arms, all models, all instances, in a public table; every pre-specified contrast whether or not significant; every deviation from this plan, with reasons, in a dedicated section.

VI. Threats and limitations

Single-shot editing is not an agent loop; the study measures context, not agency, and says so. Three models and one language. Gold localisation is a ceiling; the retrieval arms are the realistic floor. Task text may leak the fix despite screening; results are reported by source and the parametric-memory arm bounds the effect. Padding from the same repository can raise coverage; the similarity ceiling and the two other padding types address this. Temperature zero is not deterministic across providers [14], and the frontier models offer no such control; the pilot measured 7–25% non-unanimous cells at default sampling and the margin rule above is built on per-instance variance rather than on that rate. The pilot's write-sets were mostly single packages with a median of seven changed lines, which is why two-package instances are oversampled here.

Rewritten task texts are produced by a model that may also be evaluated; they are hand-checked and results are reported by task-text source. Coverage is checked syntactically through expressibility; semantic needs beyond identifiers are not captured. The forward-slice question is likely under-powered and is labelled descriptive.

VII. Timeline and resources

Mining, screening, and slices: weeks 1 to 5 (about 50 hours of manual screening; the pilot's miner yields one valid instance per eleven commits). Harness: complete and released with the pilot. Stage 1 submission: 20 November 2026. Confirmatory run after in-principle acceptance: three weeks. Analysis and Stage 2: by the EMSE deadline. Budget, from the pilot's measured unit costs ($0.20 per instance for nine arms at ten samples on the dense model; $1.22 per instance for five arms at three samples on a mid-tier model; about $8 per instance per arm at 950k tokens on a frontier model): about $700 for the design above, with an upper bound of $1,500 if long-context pricing tiers apply; verification on a laptop. All code, data, and prompts are released under Apache-2.0 and CC-BY-4.0.

AI-use disclosure

Anthropic's Claude (Fable 5.1, referred to by the author as Lemma) was used under the author's direction to draft this report, to write the analysis, mining and harness scripts, to run the pilot against the author's API account, and to cross-check references; separate Claude Opus agents produced four rounds of critical review of the design and of the pilot paper, whose findings the author adopted or rejected. The author reviewed all text and takes full responsibility for it. Generative AI tools are not authors.

References

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[2] M. Levy, A. Jacoby, and Y. Goldberg, "Same task, more tokens: The impact of input length on the reasoning performance of large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, 2024, pp. 15 339–15 353.

[3] Y. Du, M. Tian, S. Ronanki, S. Rongali *et al.*, "Context length alone hurts LLM performance despite perfect retrieval," arXiv:2510.05381, 2025.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025, describes the Agentless approach.

[5] R. Bairi, A. Sonwane, A. Kanade, V. D. C., A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, "CodePlan: Repository-level coding using LLMs and planning," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 675–698, 2024.

[6] J. C. Johansen, "ε-sufficient context for bounded readers: Definitions and a non-monotonicity result for code editing," Zenodo, version 1.1, 2026.

[7] ——, "Interfaces or bodies? a 24-instance pilot measurement of ε-sufficient context for language-model code editing in Go," Zenodo, Oct. 2026, pilot report v1.0; data and harness at https://github.com/Shockware-Limited/suffbench, tag v0.1.1-pilot.

[8] B. Sam-Bodden, "What context does a coding agent actually need to act?" arXiv:2607.09691, 2026.

[9] B. Mohammadi, L. Klein, A. Chadha, A. Arora *et al.*, "The working set of a coding agent: Coherence debt in repository-scale tasks," arXiv:2608.16630, 2026.

[10] Y. Xue, "When and how context rot appears in coding agents: A white-box study of agent skills in code auditing," arXiv:2607.17937, 2026.

[11] P. Alonso, S. Yovine, and V. A. Braberman, "TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis," arXiv:2603.17973, 2026.

[12] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing code context for LLM-based issue resolution," arXiv:2603.28119, 2026.

[13] L. Zhang, Y. Li, J. Li, X. Xia *et al.*, "Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code LLMs," arXiv:2406.18294, 2024.

[14] B. H. Bjarnason, A. Silva, and M. Monperrus, "On randomness in agentic evals," arXiv:2602.07150, 2026.

[15] B. Qin and Y. Xie, "Agent retrieval bench: Evaluating repository context retrieval for coding agents," arXiv:2607.24882, 2026.

[16] H. Li, L. Zhu, B. Zhang, R. Feng *et al.*, "ContextBench: A benchmark for context retrieval in coding agents," arXiv:2602.05892, 2026.

[17] D. Zan, Z. Huang, W. Liu, H. Chen *et al.*, "Multi-SWE-bench: A multilingual benchmark for issue resolving," arXiv:2504.02605, 2025.

[18] J. C. Johansen, "SuffBench: Specification for a context-sufficiency benchmark for code editing (v0.1.0-spec)," Shockware Limited, https://github.com/Shockware-Limited/suffbench, 2026.